

Ampliación de Prod Stage 2

Despliegue remoto con Docker, HTTPS, memoria técnica e integración continua

Profesor: Diego Ayala Bernal · Coordinador de los ciclos de Informática

Punto de partida: tu entrega de Compose — Prod Stage 2. **Modalidad:** proyecto individual o por equipos, según la organización del profesor. **Entrega:** fecha comunicada en la plataforma del módulo.

Objetivo

Continúa en el mismo repositorio y despliega la aplicación **con Docker Compose dentro del servidor online que Miralmonite tiene contratado a un proveedor externo**. No debes crear una aplicación nueva ni contratar otra cuenta de alojamiento.

Mantendrás Nginx, Unicorn y Django, el servicio de archivos estáticos y multimedia y el almacenamiento persistente. Añadirás HTTPS, documentación generada a partir del código y comprobaciones automáticas mediante integración continua. La memoria incluirá también el **trabajo de investigación obligatorio de los apartados 4.4 y 4.5**. En la defensa final aplicarás esta ampliación a tu aplicación propia.

Dónde se ejecuta cada cosa. Docker Engine y los contenedores de la aplicación se ejecutan en el servidor remoto, no en el portátil. Tu ordenador se utiliza para editar, acceder por SSH y comprobar la web. No se requiere Docker dentro de otro contenedor ni crear una máquina virtual adicional.

Arquitectura de la entrega: navegador → Nginx con HTTPS → Unicorn → Django. Nginx y la aplicación pertenecen al proyecto de Compose del servidor; la base de datos y los archivos persistentes mantienen la organización del proyecto anterior. [1]

1. Entorno asignado y normas de uso

El profesor facilitará el acceso autorizado y confirmará la compatibilidad del servidor con Docker. No reinstales Docker ni alteres la configuración general sin autorización. Cada equipo trabajará únicamente en su espacio, con los puertos y recursos asignados.

Dato que facilita el profesor	Uso en la práctica
Dirección del servidor y acceso SSH	Acceder al equipo remoto con la cuenta o procedimiento autorizado.
Carpeta y nombre del proyecto Compose	Distinguir tu despliegue de los demás; por ejemplo, dapw_equipo01.
Subdominio real y puertos HTTP/HTTPS	Comprobar la web y configurar el certificado sin ocupar puertos ajenos.
Recursos y modalidad de acceso a Docker	Respetar límites y turnos; utilizar el motor autorizado o ejecución supervisada.

Usa datos ficticios. No publiques información personal real ni credenciales del colegio. No detengas servicios ajenos, no reinicies el servidor y no ejecutes limpiezas globales de Docker.

2. Ejecutar Compose en el servidor

2.1. Preparar la versión remota

Accede por SSH, comprueba que estás en el servidor y descarga tu repositorio dentro de la carpeta asignada. Identifica el commit que desplegarás. Documenta el proveedor y la modalidad de servicio que te confirme el profesor, el sistema operativo, la arquitectura, CPU, RAM, disco y recursos asignados, así como las versiones de Docker Engine y Compose. **No inventes datos ni muestres información confidencial.**

Prepara un **archivo completo de producción**, por ejemplo `compose.prod.yaml`, a partir de Prod Stage 2. Mantén el código de la aplicación dentro de la imagen y retira los montajes y la recarga automática destinados al desarrollo. [1]

Elemento	Requisito que se mantiene o se adapta
Django y Unicorn	Arranque con Unicorn, no con runserver; <code>DEBUG=False</code> , hosts autorizados y secretos fuera del código versionado.
Nginx	Entrada HTTP/HTTPS del proyecto; rutas de estáticos y multimedia y proxy a Unicorn.
Red de Compose	Unicorn y la base de datos no se publican directamente al exterior. Nginx usa el nombre interno del servicio.
Estáticos y multimedia	Compila Tailwind cuando corresponda, ejecuta <code>collectstatic</code> y comprueba los montajes compartidos; Nginx solo necesita lectura.
Base de datos	Conserva la utilizada en la práctica y asegura la persistencia. No se exige cambiar de gestor.
Convivencia entre equipos	Nombre Compose propio; puertos exclusivos; sin nombres globales fijos de contenedor o volumen que colisionen con otros proyectos.

Un nombre de proyecto diferente organiza los recursos de Compose, pero **no sustituye el aislamiento de permisos** del servidor. El administrador establece qué motor y recursos puedes utilizar. [2, 3]

2.2. Arrancar y verificar

Ejecuta los comandos por SSH en el **servidor remoto**. Adapta el nombre del proyecto y el archivo a los asignados; el siguiente ejemplo presupone un archivo de producción completo:

```
docker compose -p dapw_equipo01 -f compose.prod.yaml config --quiet
docker compose -p dapw_equipo01 -f compose.prod.yaml up -d --build
docker compose -p dapw_equipo01 -f compose.prod.yaml ps
docker compose -p dapw_equipo01 -f compose.prod.yaml logs --tail=50
```

Comprueba que la aplicación responde desde otro ordenador **con los contenedores de tu portátil detenidos**. Verifica páginas, estilos, imágenes y una operación representativa. Revisa los logs básicos para resolver incidencias, sin compartir secretos en capturas.

Crea un dato o archivo de prueba y recrea únicamente tus contenedores **sin eliminar volúmenes**. Demuestra que la información permanece. No utilices `down -v` para esta comprobación ni borres la base de datos.

Evidencias: versión desplegada, estado de los servicios remotos, acceso desde otro equipo, operación funcional y comprobación de persistencia.

3. Activar HTTPS en tu Nginx

3.1. Obtener el certificado

Utiliza el **subdominio real autorizado** por el profesor y comprueba que resuelve al destino previsto. No uses un nombre .test para solicitar un certificado público. La gestión global del dominio y del servidor corresponde al administrador.

Obtén un certificado para tu subdominio mediante **Let's Encrypt y Certbot, con supervisión del profesor**. Participa en la solicitud y documenta la validación. El nombre de acceso debe figurar en el certificado. Conserva la clave privada únicamente en el espacio protegido autorizado.

Servidor compartido. Para no ocupar el puerto 80 de otras aplicaciones, el profesor puede organizar la validación DNS-01: generarás la solicitud y facilitarás el registro TXT requerido para que el administrador lo publique. No recibirás las credenciales generales del DNS. HTTP-01/webroot solo se utilizará si el administrador ha habilitado la ruta pública de validación en el puerto 80. [4, 5]

Cuando se asignen puertos externos distintos de 80/443, utilizarás esos puertos para visitar la aplicación. El certificado identifica el nombre del servidor; en las URL y redirecciones debe figurar el puerto HTTPS externo correcto. No reserves 80/443 por tu cuenta.

3.2. Configurar y comprobar

Configura **HTTPS en el contenedor Nginx de tu proyecto**, montando el certificado, su cadena y la clave privada de forma segura. No basta con que un panel o proxy ajeno añada HTTPS sin que tú configures y demuestres tu servidor. [6]

Mantén /static/, /media/ y el proxy a Unicorn. Añade la redirección de HTTP a la dirección HTTPS autorizada, sin romper el método de validación elegido. No incorpores claves a la imagen, a Git ni al ZIP.

Revisa en Django la detección de HTTPS detrás de Nginx, las cookies seguras y los formularios. Confía en X-Forwarded-Proto solo cuando lo establezca el proxy controlado; revisa los orígenes CSRF cuando sea necesario, sin comodines indiscriminados. [7]

Comprobación	Evidencia que debes entregar
Conexión HTTPS	Web abierta desde otro equipo sin advertencias de identidad o confianza.
Certificado	Nombre, emisor y fechas de validez; ninguna captura de la clave privada.
Redirección	Acceso HTTP que termina en la URL HTTPS y el puerto correctos.
Recursos y formularios	Estilos e imágenes sin contenido mixto; una operación de la aplicación completada.
Instalación y mantenimiento	Fragmentos de Nginx, rutas de montaje y procedimiento de renovación/recarga.

Renovación: registra la fecha de caducidad y explica cómo renovarías e instalarías el certificado y recargarías Nginx. Una validación DNS manual no se renueva automáticamente sin automatización adicional; no declares una renovación automática que no hayas configurado. [5]

4. Memoria técnica y control de versiones

4.1. Elegir y utilizar un generador

Identifica **dos herramientas de generación de documentación**, explica su finalidad y justifica cuál utilizarás. Se propone **Sphinx con autodoc**; otra herramienta requerirá aprobación y deberá generar documentación de componentes del código, no limitarse a páginas escritas manualmente.

Crea docs/ y genera documentación HTML. Incorpora docstrings útiles y documenta automáticamente **al menos dos componentes reales y relevantes** de tu aplicación. No basta con copiar el código en una página. Sphinx/autodoc permite incorporar documentación de módulos, clases y funciones. [8]

Apartado de la documentación	Contenido mínimo
Servidor y arquitectura	Ficha del servidor, recursos asignados y componentes de la aplicación: Nginx, Unicorn, Django, red, base de datos y almacenamiento.
Instalación y despliegue	Dependencias, configuración y pasos reproducibles para ejecutar Compose en el servidor contratado.
HTTPS	Solicitud e instalación del certificado, pruebas y procedimiento de sustitución/renovación.
Administración	Actualizar, detener y arrancar el proyecto; registros básicos y datos que deben conservarse.
Referencia del código	Documentación generada de dos componentes reales a partir de sus docstrings.

Mantén el **README.md como entrada al repositorio** y explica cómo reconstruir el HTML desde sus fuentes. La memoria debe reunir la documentación de lo implementado y los apartados de investigación **4.4 y 4.5**. No entregues un cuestionario separado ni dupliques explicaciones: organiza y enlaza los contenidos dentro de docs/.

4.2. Mostrar un uso real de Git

Incluye evidencias de la disponibilidad o instalación de Git, su configuración y las operaciones utilizadas: clonar, crear una rama, registrar cambios y sincronizar el repositorio. El historial debe permitir reconocer la evolución de la ampliación; no se exige un número artificial de commits.

Realiza una modificación de documentación en una rama y abre una **pull request**. Un compañero o el profesor deberá revisarla antes de incorporarla a la rama principal. Conserva el intercambio y la resolución como evidencia de trabajo colaborativo.

4.3. Proteger y documentar el repositorio

Revisa permisos y acceso del profesor. Configura `.gitignore` y `.dockerignore`; proporciona un `.env.example` con valores ficticios. Excluye claves privadas, tokens, contraseñas, archivos de configuración ACME sensibles y bases de datos con información personal.

La exclusión debe comprobarse también en el historial y en el contexto de construcción. No publiques la clave de producción de Django; utiliza configuración externa y datos de prueba. [7]

Añade un apartado que permita a otra persona **instalar/configurar Git, obtener el repositorio, crear una rama, registrar y sincronizar cambios, revisar una modificación e identificar la versión entregada**. Distingue estas operaciones de los comandos de Docker.

4.4. Documentar y justificar el despliegue

Desarrolla estos **seis apartados dentro de la memoria**, aplicados a tu propio proyecto. Relaciona las explicaciones con archivos, comandos y evidencias de la entrega. Puedes enlazar a lo ya explicado en instalación o HTTPS, sin repetirlo.

A. Servidor contratado y comparación con la nube

Elabora la ficha del servidor con los datos autorizados del apartado 2.1. Identifica si el servicio es una VM cloud, un VPS, un dedicado u otra modalidad y justifica la identificación con la información facilitada; si falta un dato, indícalo como no facilitado. Compara el entorno con una VM de Oracle Cloud: qué conservarías de Compose y qué revisarías al cambiar de proveedor. **No debes contratar ni desplegar en Oracle.** [10, 11]

B. Evolución de la arquitectura

Compara Dev Stage, Prod Stage 1 y Prod Stage 2. Explica qué problema resuelve cada etapa y qué ventajas e inconvenientes tiene añadir Nginx delante de Unicorn. Relaciona las diferencias con los servicios, los comandos de arranque y los archivos de configuración de tu proyecto.

C. Recorrido de una petición HTTP/HTTPS

Describe el recorrido desde el navegador hasta Django y la respuesta de vuelta. Indica dónde termina HTTPS y cómo se comunica Nginx con Unicorn en tu configuración. Distingue una petición a la aplicación de una petición a /static/ o /media/ e identifica los servicios y puertos que intervienen.

D. Direcciones, nombres y puertos

Explica qué significa localhost en el portátil, en el servidor contratado y dentro de cada contenedor. Justifica por qué web:8000 no puede sustituirse siempre por localhost:8000. Identifica los nombres reales de tus servicios, los puertos internos, los publicados y las direcciones que utiliza el navegador.

E. Investigación de incidencias y error 502

Redacta un procedimiento ordenado para investigar un error 502 en tu despliegue: servicios, nombres, puertos y registros que revisarías, y comandos que utilizarías. Documenta las incidencias reales que hayan surgido y su solución. Si no has observado ese error, presenta el procedimiento como **hipótesis de diagnóstico**, no como una incidencia que hayas resuelto.

F. Manual de mantenimiento

Explica qué necesita otra persona para mantener tu despliegue: actualizar la aplicación, sustituir o renovar un certificado y actuar ante problemas de almacenamiento. Identifica qué archivos, volúmenes y datos deben conservarse. Relaciona los pasos con tu proyecto y distingue las operaciones que puedes realizar de las que requieren autorización del administrador.

Vincula esta parte a la versión entregada: las rutas, nombres, puertos y evidencias deben coincidir con el despliegue que mostrarás. Oculta claves, tokens, contraseñas y datos personales.

4.5. Investigar mejoras del despliegue

Desarrolla también estos **seis apartados obligatorios**, con fuentes y propuestas aplicadas a tu servidor. En esta ampliación **no se exige implementar estas mejoras ni ejecutar pruebas de carga**: debes estudiarlas y explicar cómo las realizarías. Identifica claramente lo propuesto y no inventes resultados.

G. Módulos del servidor y compresión gzip

Define qué es un módulo de un servidor web. Investiga cómo añadirías compresión gzip a tu Nginx, qué recursos tendría sentido comprimir y cómo comprobarías su efecto. Explica la configuración que propones y relaciona el ejemplo con los archivos que sirve tu aplicación.

H. Varios sitios en un mismo servidor

Propón cómo alojarías dos aplicaciones con nombres distintos en una misma dirección IP. Describe los cambios en los bloques de Nginx, el uso de puertos y la relación entre nombres y certificados HTTPS. Distingue tu propuesta de los puertos y del acceso que realmente te haya asignado el profesor.

I. Autenticación y control de acceso

Diferencia la autenticación de Django de la autenticación del servidor web. Investiga cómo protegerías una zona de documentación desde Nginx y qué necesitarías configurar. Describe las comprobaciones previstas con un usuario autorizado y otro no autorizado, sin publicar credenciales reales.

J. Gestión y análisis de logs

Compara la consulta de docker logs con un sistema de gestión de registros. Propón cómo recogerías y reunirías los logs de Nginx y de la aplicación y qué información analizarías. Relaciona la propuesta con los registros básicos que sí has utilizado durante el despliegue.

K. Diseño de una prueba de rendimiento

Distingue las pruebas funcionales que has ejecutado de una prueba de rendimiento. Diseña **por escrito** una prueba pequeña para tu aplicación: objetivo, duración, usuarios, métricas y precauciones. No la ejecutes contra el servidor compartido ni presentes medidas o conclusiones experimentales que no hayas obtenido.

L. Diagnóstico de una aplicación lenta

Analiza el supuesto de una web que funciona con un usuario, pero responde lentamente con varios. Explica qué investigarías antes de aumentar la RAM, relacionando Unicorn, CPU, memoria, base de datos y tiempos de respuesta. Indica qué datos necesitarías para justificar una decisión, sin atribuir al servidor un problema no observado.

Fuentes y elaboración

Redacta con tus propias palabras y cita la documentación consultada en cada apartado: organización o autor, título, enlace y fecha de consulta. Prioriza fuentes oficiales; puedes partir de los recursos del final. Explica cómo se aplica la información a tu proyecto. **Una captura no sustituye la explicación ni una propuesta sustituye una comprobación realizada.**

5. Integración continua con GitHub Actions

Crea un flujo en `.github/workflows/ci.yml` que se ejecute al enviar cambios y al abrir o actualizar una pull request. Las comprobaciones se ejecutarán en el **entorno de GitHub Actions**, no en el servidor de Miralmonte. [9]

Comprobación automática	Resultado esperado
Entorno	Instala las dependencias necesarias con configuración y datos de prueba.
Imagen Docker	Comprueba que se puede construir la imagen de la aplicación.
Pruebas reales	Ejecuta al menos dos pruebas relevantes con resultados verificables: una ruta y una funcionalidad o validación, por ejemplo.
Documentación	Genera el HTML y lo conserva como artefacto recuperable de la ejecución.

No sirve un flujo que únicamente imprima un mensaje de éxito. Muestra una ejecución que detecte un fallo controlado en una rama de trabajo y otra satisfactoria tras corregirlo. Explica qué falló y qué cambio resolvió el problema.

No se pide despliegue automático. El acceso al servidor y la ejecución de Compose siguen realizándose por el procedimiento autorizado. No entregues a GitHub Actions las credenciales administrativas del colegio ni configures un runner de la clase en ese servidor.

6. Entrega y defensa

Actualiza el mismo repositorio de Prod Stage 2 y marca la versión con una etiqueta identificable. Entrega el **enlace al repositorio, un ZIP depurado y la memoria completa**, con las evidencias y la investigación. Las credenciales de prueba se comunicarán por el canal privado indicado.

Material	Contenido
Despliegue	Dockerfile, Compose de producción y configuración de Nginx adaptada al servidor remoto.
Configuración de ejemplo	Variables y rutas necesarias con valores ficticios; sin secretos ni claves privadas.
Memoria técnica e investigación	README, fuentes de docs/ y HTML o artefacto. Incluye los 12 apartados A-L de 4.4 y 4.5, las evidencias y las fuentes citadas.
Git e integración continua	Historial, revisión de documentación, workflow, pruebas y ejecuciones comentadas.
Funcionamiento	Acceso remoto con el portátil detenido, HTTPS, operación funcional y persistencia.

En la defensa mostrarás el despliegue remoto y explicarás tu memoria. El profesor podrá pedirte localizar una configuración, justificar una decisión o ampliar una conclusión de la investigación. **Los apartados A-L deben estar entregados por escrito; no basta con responderlos oralmente.**

Alcance: son prácticos el despliegue remoto, HTTPS, la documentación generada, Git y CI. Los apartados de mejora de 4.5 son investigación escrita. No se añade una práctica específica de RA4 o RA5 ni se modifican las ponderaciones comunicadas.

Recursos oficiales de consulta

[1] Docker. Compose en producción.

[2] Docker. Nombres de proyecto Compose.

[3] Docker. Privilegios del grupo docker y modo rootless.

[4] Let's Encrypt. Validaciones HTTP-01 y DNS-01.

[5] Certbot. Obtención, instalación y renovación.

[6] Nginx. Configuración de servidores HTTPS.

[7] Django 5.2. Lista de comprobación del despliegue.

[8] Sphinx. Generación con autodoc.

[9] GitHub Actions. Construcción y pruebas de Python.

[10] NIST SP 800-145. Definición de cloud computing.

[11] Oracle Cloud Infrastructure. Compute.